ODYSS

Documentación del proyecto backend

Guía de arquitectura, modelo de dominio, flujos principales, persistencia y pruebas del proyecto de planificación de viajes.

Propósito: explicar cómo se organiza el backend y cómo se conectan sus conceptos, desde el modelo TDM hasta los servicios y la base de datos.

Proyecto HARIA · .NET 9 · NHibernate · SQLite

Documento preparado a partir de los archivos actuales del repositorio. Fecha: 4 de noviembre de 2026.

1. Resumen ejecutivo

Odyss es un backend para organizar viajes y la información asociada a cada viajero. El modelo contempla usuarios, viajes con varios destinos ordenados, actividades, archivos multimedia, recordatorios, tareas, reseñas y estadísticas.

El modelo conceptual está definido en **DSM_Odyss.tdm**. HARIA genera el dominio y buena parte de la aplicación e infraestructura. La solución usa NHibernate para guardar el modelo en SQLite y separa el acceso a datos de los servicios de aplicación.

En el repositorio inspeccionado no aparece un proyecto web ni una API HTTP. El backend expone servicios/casos de uso .NET y persistencia; una interfaz cliente o API de red sería una capa adicional.

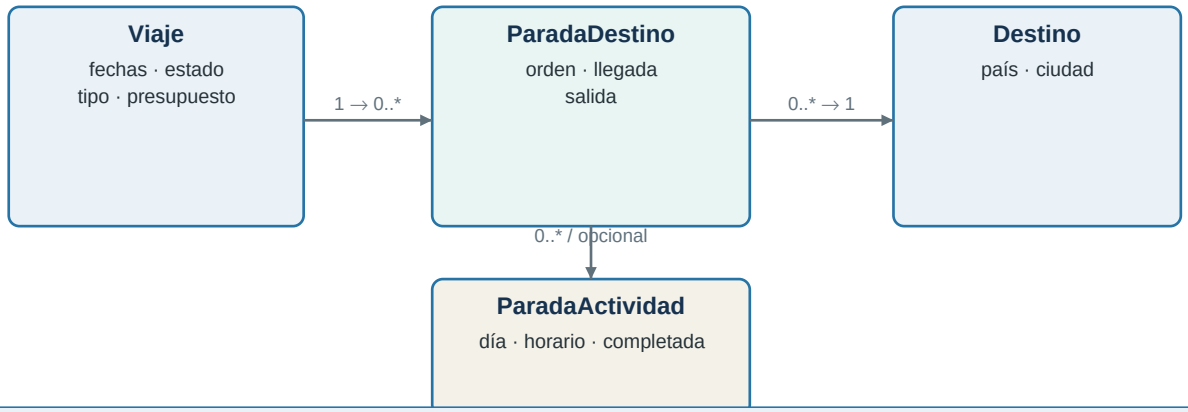
Qué contiene

- **Modelo:** 14 clases de dominio y 5 enumeraciones en el TDM.
- **Operaciones:** CRUD por entidad, filtros, login, operaciones personalizadas y transacciones de aplicación.
- **Persistencia:** NHibernate, mapeos XML y SQLite.
- **Pruebas:** xUnit generado y manual, incluyendo itinerarios con varios destinos.

2. Modelo del dominio

El dominio distingue entre el **catálogo de lugares** y la **visita a un lugar en un viaje específico**. Así se reutiliza “Roma” en varios viajes y se registran fechas y posiciones diferentes.

Cada parada es una visita del itinerario, no un lugar duplicado.



Ejemplo: un viaje puede tener estas paradas: 1) Madrid, 2) París, 3) Roma. Cada parada apunta a un Destino y conserva sus propias fechas. Una actividad puede asociarse opcionalmente a una parada concreta.

Entidades principales

Entidad	Responsabilidad en el dominio
Cliente	Cuenta de usuario, datos personales, preferencias de notificación y relaciones. Es el tipo User del modelo.
Viaje	Itinerario: título, fechas, presupuesto, tipo, estado, autor y visibilidad.
Destino	Lugar del catálogo, con país y ciudad opcional. No lleva fechas del itinerario.
ParadaDestino	Visita de un viaje a un destino: orden, llegada y salida.
ParadaActividad	Actividad de un día/hora; puede estar completada y asociarse opcionalmente a una parada de destino.
Ubicacion	Lugar geográfico con nombre y coordenadas, vinculable a actividades.
Carpeta	Contenedor de multimedia relacionado con viajes y que puede compartirse.
Multimedia	Metadatos de fotos/vídeos guardados en una carpeta.
Collage	Collage asociado a un viaje, con URL de imagen y fecha.
Checklist / Tarea	Lista de preparación del viaje y elementos marcables como completados.
Recordatorio	Aviso con fecha, hora, tipo y estado de envío; asociado a usuarios y opcionalmente a una tarea.
Resena	Comentario y puntuación de un viaje escrita por un cliente.
Estadisticas	Resumen anual de países visitados, fotos y destino más visitado por cliente.

Relaciones destacadas

- Un **Viaje** contiene muchas **ParadaDestino**; cada parada pertenece a un viaje y apunta a un Destino.
- Una **ParadaDestino** puede tener varias actividades; una actividad puede no tener parada asignada.
- Un **Cliente** es autor de viajes y reseñas, puede tener contactos y recibir recordatorios.
- Las carpetas agrupan multimedia; las checklists agrupan tareas.
- El TDM expresa cardinalidades y navegabilidad junto con clases y operaciones.

3. Vocabulario del itinerario

Concepto	Qué representa	Ejemplo
Destino	El sitio reutilizable del catálogo.	París, Francia
ParadaDestino	Una visita concreta dentro de un viaje, con orden y fechas.	Viaje de verano, parada 2, París, 10–13 julio
ParadaActividad	Algo que se hace durante el viaje; se puede asignar a una parada.	Museo, día 2, 11:00

Esta separación evita que el objeto del catálogo cargue datos que dependen de cada viaje. Si dos viajes visitan París en fechas distintas, ambos pueden referirse al mismo Destino y tener cada uno su ParadaDestino.

4. Organización del backend

Proyecto	Responsabilidad
DSM_Odyss.Domain	Entidades y tipos de dominio, como Viaje, Cliente, Destino y ParadaDestino.
DSM_Odyss.Application	Servicios, contratos/repositorios abstractos, casos de uso y excepciones de modelo.
DSM_Odyss.Infrastructure	Implementaciones NHibernate, repositorios, Unit of Work, mapeos XML, seguridad y logging.
DSM_Odyss.Tests	Pruebas automatizadas xUnit: comportamiento generado y operaciones manuales.
InitializeDB	Programa de inicialización del esquema y datos iniciales.

Dependencias y recorrido típico

La aplicación depende del dominio y de abstracciones, no de consultas directas a SQLite. Infrastructure implementa los repositorios y se encarga de NHibernate. Unit of Work reúne repositorios, sesión y operaciones transaccionales.

- **Servicio:** aplica una operación de negocio, por ejemplo cambiar estado o validar una parada.
- **Repositorio:** consulta y persiste entidades; algunas consultas especializadas usan NHibernate LINQ.
- **Unit of Work:** agrupa el acceso a datos y abre, confirma o revierte transacciones.
- **UseCase:** coordina una operación de aplicación como duplicar el itinerario.

HARIA genera servicios y estructura base a partir del TDM. El código manual extiende operaciones de negocio. Tras regenerar, conviene revisar que las extensiones y mapeos personalizados sigan conservados.

5. Operaciones del modelo

Tipo HARIA	Uso
New	Crear una entidad.
Destroy	Eliminar una entidad.
Modifier	Modificar sus datos.
ReadOID	Leer una entidad por identificador.
ReadAll	Listar entidades.
ReadFilter	Consultar entidades con criterio tipado, por ejemplo, viajes públicos.
Login	Autenticar el tipo User (Cliente) y producir el resultado/token.
Custom	Operación de negocio adicional de un servicio.
CustomTransaction	Operación coordinada con transacción desde un UseCase.
Relationer / Unrelationer	Crear o retirar asociaciones; roles múltiples reciben listas de OID.

Conteo del TDM actual: las 14 clases tienen New/Destroy/Modifier/ReadOID/ReadAll (14 de cada tipo); hay 16 filtros, 18 Custom, 3 CustomTransaction, un Login, 19 Relationer y 6 Unrelationer. Son declaraciones del modelo, no una garantía de cobertura de todo escenario externo.

Filtros incluidos

Incluyen viajes públicos y por estado, paradas por viaje, actividades pendientes, destinos por país, ubicaciones por nombre, recordatorios pendientes, tareas por estado, reseñas por puntuación, collages por año, carpetas con multimedia y checklists con tareas.

6. Funcionalidades por área

Viajes e itinerarios

El viaje contiene título, fechas, presupuesto, tipo, estado y visibilidad pública. Puede asociarse a varias paradas de destino y actividades. Hay operaciones para compartir/publicar itinerarios, cambiar el estado y duplicarlos.

Duplicar itinerario

La duplicación crea un viaje nuevo y copia sus paradas de destino y actividades. Consulta explícitamente los repositorios, sin depender de colecciones NHibernate que quizá no estén cargadas. La copia se inicia privada y planificada; las actividades copiadas quedan pendientes.

Paradas y reglas

El servicio valida orden positivo y único dentro del viaje, llegada no posterior a salida y fechas dentro del intervalo del viaje. La consulta por viaje ordena resultados por orden.

Usuarios y colaboración

Cliente centraliza login, cambio de contraseña, preferencias de notificación, contactos y relaciones con viajes, reseñas y estadísticas. Viajes y carpetas incluyen operaciones de compartir y dejar de compartir.

Contenido, tareas y avisos

Las carpetas organizan metadatos multimedia; las tareas pertenecen a checklists; los recordatorios almacenan cuándo avisar y si se han enviado. Una operación de notificación no implica que haya un proveedor externo de email/SMS configurado.

Estadísticas, collage y reseñas

El modelo prevé recap anual, países coloreados, estadísticas recalculables, generación de collages y reseñas puntuadas. Las estadísticas consideran los destinos asociados a las paradas del itinerario.

7. Persistencia y configuración

- **ORM:** NHibernate 5.5.2 en proyectos .NET 9.
- **Base de datos:** SQLite (System.Data.SQLite.Core 1.0.119 en pruebas).
- **Mapeos:** archivos XML *.hbm.xml en Infrastructure/Mappings/ModeloDeDominio_Odyss.
- **Configuración HARIA:** haria.config.json define ORM, base, framework, proyecto y carpeta generated.
- **Configuración de ejecución:** InitializeDB/appsettings.json y hibernate.cfg.xml definen conexión y logging; también se cargan variables de entorno.
- **Transacciones:** UnitOfWork ofrece begin/commit/rollback; la infraestructura puede capturar SQL para logging.

Inicialización local

InitializeDB construye la configuración NHibernate, crea el esquema con SchemaExport e inserta datos iniciales generados y manuales. La conexión configurada usa un archivo SQLite DSM_Odyss.db. Antes de inicializar, comprueba la ruta de trabajo y conserva la base existente si se necesita.

Seguridad

Cliente está marcado como User y almacena una Password del modelo. Infrastructure registra implementaciones HARIA de hash de contraseña y generación de tokens; el login usa usuario y contraseña. Si se añade una API HTTP, también harán falta controles de seguridad en esa capa.

8. Pruebas y estado de validación

El proyecto usa xUnit y contiene casos generados y manuales. Los casos manuales inspeccionados comprueban filtros, contactos y carpetas compartidas, cambio de contraseña/login, preferencias, estados y transacciones para vaciar checklists y duplicar viajes con varias paradas.

Resultado observado con dotnet test en este entorno: 25 pruebas totales, 21 correctas y 4 fallidas. Las cuatro fallidas terminan al intentar cargar `SQLite.Interop.dll`; Windows devuelve 0x800700CE (“el nombre del archivo o la extensión es demasiado largo”). No debe interpretarse como una validación funcional completa. La guía no atribuye el error a una regla concreta del dominio.

La prueba de itinerarios comprueba dos destinos con órdenes distintos, rechazo de orden duplicado, asociación opcional de actividad, copia de destinos y estado inicial de la copia. Conviene volver a ejecutar la suite desde una ruta de trabajo más corta o con la configuración SQLite corregida.

9. Estructura útil del repositorio

Ruta relativa	Contenido
DSM_Odyss/DSM_Odyss.tdm	Fuente conceptual HARIA del dominio y operaciones.
DSM_Odyss/haria.config.json	Configuración de generación HARIA.
DSM_Odyss/generated/DSM_Odyss.Domain	Entidades y enums generados.
DSM_Odyss/generated/DSM_Odyss.Application	Servicios, abstracciones, casos de uso y extensiones manuales.
DSM_Odyss/generated/DSM_Odyss.Infrastructure	Repositorios, UnitOfWork, mapeos y configuración de datos.
DSM_Odyss/generated/DSM_Odyss.Tests	Pruebas de modelo y operaciones.
DSM_Odyss/generated/InitializeDB	Herramienta para esquema y datos iniciales.

10. Glosario y decisiones de diseño

Término	Significado en este proyecto
TDM	Modelo de dominio textual usado por HARIA para generar la solución.
OID	Identificador que enlaza una operación o entidad relacionada.
Cardinalidad	Cantidad permitida de elementos relacionados.
ReadFilter	Lectura de entidades que cumple una condición.
Unit of Work	Componente que agrupa repositorios y coordina operaciones de base de datos.
NHibernate mapping	Reglas XML para mapear objetos a tablas/relaciones.
CustomTransaction	Operación cuyo UseCase coordina commit o rollback.

Decisiones que conviene recordar

- Destino es un lugar del catálogo; ParadaDestino es una visita contextual dentro de un viaje.
- Orden y fechas pertenecen a cada parada, así un destino se reutiliza en viajes diferentes.
- Las actividades pertenecen a un viaje y su asociación a una parada concreta es opcional.
- La guía describe el backend presente; la autenticación y permisos de red requieren una API si se necesita acceso remoto.

Fuentes inspeccionadas

DSM_Odyss/DSM_Odyss.tdm · DSM_Odyss/haria.config.json · generated/DSM_Odyss.Domain · generated/DSM_Odyss.Application · generated/DSM_Odyss.Infrastructure · generated/DSM_Odyss.Tests · generated/InitializeDB.

Documento basado en los archivos revisados el 4 de noviembre de 2026. El código generado puede cambiar al volver a ejecutar HARIA.